Parallel and Asynchronous Simulation of Pandemic Spread

CSE600 – Assignment 3

October 30, 2026

1 Overview

You need to build a simulation of an infectious disease spreading through a population, and then use it to study parallel and asynchronous execution. The challenges are:

- parallelising it *correctly*, so that results are identical for any number of threads;
- using asynchronous programming to overlap computation with slow work;
- carrying out a careful study of performance at several levels of parallelism.

No locks, semaphores or condition variables are required. The only coordination you need is starting tasks and waiting for them to finish (thread `join`, or waiting on a future).

2 The Model

2.1 Population and states

The population is an $N \times N$ grid with one person per cell. Each person is in one of four states:

- **S** (Susceptible): healthy, can be infected.
- **I** (Infected): sick and contagious. Also stores c, the number of days infected.
- **R** (Recovered): immune.
- **V** (Vaccinated): immune
- **D** (Dead).

Each person's neighbours are the eight surrounding cells. Cells outside the grid do not exist; the edges do not wrap around.

2.2 Update rules

The simulation advances one day at a time. All cells are updated together: the state on day $t + 1$ depends only on the state on day t.

1. **Infection.** A susceptible person with k infected neighbours becomes infected with probability
$$P(k) = 1 - (1 - \beta_{\text{eff}})^k$$
where β_{eff} is the current transmission probability. A newly infected person starts with $c = 0$.
2. **Progression.** An infected person's counter c increases by one. When it reaches d, the person dies with probability μ, and otherwise recovers.
3. Recovered and dead people never change state.

2.3 Initial conditions

First, a fraction v of the population, chosen at random, is vaccinated (set to V). Then I_0 susceptible people, chosen at random, are infected. This setup may be done sequentially.

2.4 Interventions

The main thread changes β_{eff} between days, using the daily totals:

- **Masks:** $\beta_{\text{eff}} = \beta \times m$ for the whole run, where $0 < m \leq 1$.
- **Lockdown:** when the infected fraction rises above θ_{on}, multiply β_{eff} by λ. Lift the lockdown when the infected fraction falls below θ_{off}. A decision made from day t's totals applies from day $t + 1$.

2.5 Default parameters

Parameter	Symbol	Default
Grid size	N	1000
Transmission probability	β	0.08
Days infectious	d	7
Mortality	μ	0.01
Initially infected	I_0	10
Vaccinated fraction	v	0
Mask factor	m	1 (off)
Lockdown factor	λ	0.3
Lockdown thresholds	$\theta_{\text{on}}, \theta_{\text{off}}$	0.02, 0.005
Random seed	s	42

A run ends when no one is infected, or after a maximum number of days. For timing experiments, use a fixed number of days so that every run does the same amount of work.

3 Concurrency Rules

1. **Two grids.** Keep a *current* grid and a *next* grid. During a day, threads read only from *current*, and each thread writes only its own cells of *next*. Swap the grids after all threads have finished. Explain in your report why this design has no data races.
2. **Each cell updates itself.** A cell decides its own next state by reading its neighbours. It never writes to a neighbour.
3. **Only join or wait.** Use only thread `join` or waiting on futures. If you use anything else (locks, atomics, barriers), justify it in your report.
4. **Local counts.** Each thread counts S, I, R, V and D in its own part of the grid and returns the counts. The main thread adds them up.

3.1 Random numbers

A single shared random generator would make results depend on thread timing. One generator per thread would make results depend on the number of threads. Neither is acceptable.

Instead, compute every random number from a hash of (seed, day, cell, stream), where *stream* separates different decisions (0 for infection, 1 for death). The result then depends only on which cell and day it is, not on which thread computes it. function is given in Appendix B.

Correctness requirement: for the same seed and parameters, your program must produce exactly the same output for every number of threads, every way of dividing the grid, and both the sequential and parallel versions.

4 Tasks

Part A: Sequential version (10 marks)

1. Implement the model as a single-threaded program.
2. Read all parameters from the command line, including grid size, number of days, seed, model parameters, number of threads, number of chunks and snapshot settings.
3. Write one line per day to a CSV file: `day,S,I,R,V,D,beta_eff,checksum`, where `checksum` is a hash of the whole grid.

Part B: Threaded version (20 marks)

1. **Thread per day.** Each day, split the grid into C strips of rows, start one thread per strip, and join them all before swapping the grids. Use lambda function where necessary.
2. **Thread pool.** Create a pool of T threads once. Each day, submit C tasks (C may be larger than T) and wait for all of them to finish.
3. **Ways of dividing the grid.** Support strips of rows, square blocks, and interleaved rows (thread i takes rows i, $i + T$, $i + 2T$, ...).
4. **Testing.** Write a script that runs the program with $T = 1, 2, 3, 4, 7, 8, 16$ and every way of dividing the grid, and checks that every CSV file is identical to the sequential one.

Commit Part-B on a single branch.

Part C: Asynchronous programming (20 marks)

C1. Background snapshot writer. Every K days, save a picture of the grid as a PPM image file (one colour per state).

1. **Blocking version:** write the file directly in the main loop.
2. **Asynchronous version:** copy the grid, submit the write as an asynchronous task, and continue simulating immediately. Explain why the copy is necessary.
3. **Limit:** allow at most Q writes to be pending. If Q are pending, wait for the oldest one to finish before submitting another.
4. Add an option `--io-latency` that adds a delay (in milliseconds) to each write, to imitate slow storage.
5. At the end, wait for all pending writes and report any that failed.

Create Part-C on its own branch.

C2. Parameter sweep. Write a program that runs many independent simulations, for example every combination of several values of β and v, each with several different seeds.

1. Submit each run as an asynchronous task that returns a summary: peak infected fraction, day of the peak, total infected, total deaths, and number of days.
2. Handle summaries *in the order they finish*, printing progress (e.g. `37/120 runs done`) and adding each to a results CSV file.
3. Support a time limit: when it runs out, cancel runs that have not started, wait for running ones, and report which runs are missing.
4. In your report, explain the difference between handling results in the order they finish and in the order they were submitted.

Part D: Performance study (20 marks)

Follow the method in Section 5. For each experiment, give a graph and explain the results.

D1. Strong scaling. Use $N = 4000$, 200 days, no snapshots. Measure the time for $T = 1, 2, 4, \ldots$ up to twice the number of logical cores. Plot speedup and efficiency. Calculate the Karp–Flatt serial fraction (Appendix C) and compare with Amdahl's law. What happens beyond the number of physical cores?

D2. Weak scaling. Keep the work per thread fixed (e.g. 1000×1000 cells per thread) and increase the grid size with T. Plot efficiency and relate it to Gustafson's law. If you do not reach the ideal, explain why (consider memory speed and caches).

D3. Size of work units.

1. With T equal to the number of physical cores, vary the number of chunks C from T to $64T$. Find the best value and explain why.
2. Compare the three ways of dividing the grid.
3. Compare thread-per-day with the thread pool for small grids ($N = 50$ to 1000). Find the smallest grid size at which the parallel version is faster than the sequential one, and explain why.

D4. Parallelism within a run vs. across runs. Using P cores (your number of physical cores), complete 32 independent runs from the sweep in C2. Try different ways of using the cores: R runs at once, each with T threads, where $R \times T = P$ (for example 1×8, 2×4, 4×2, 8×1). Measure the total time and the runs completed per second. Which is best, and why? Consider the overhead of threads within a run, runs that last different numbers of days, and memory use.

D5. Hiding I/O time. Compare the total time with no snapshots, blocking snapshots and asynchronous snapshots, for different values of `--io-latency` and K. How much of the writing time does the asynchronous version hide? How does Q affect time and memory? When does writing become the bottleneck?

Part E: Epidemiology (10 marks)

Use $N = 500$ to 1000, run until the epidemic ends, and use at least 10 seeds for each setting. Report averages and spread.

1. Plot S, I, R and D over time for the default parameters.
2. Vary β and find the value below which outbreaks die out quickly.
3. Vary v and estimate the vaccination level needed for herd immunity.
4. Show how masks and lockdown change the height and timing of the peak.
5. Solve the standard SIR equations with similar parameters and compare with your grid results. Explain the differences. (Hint: in the grid, people can only infect their neighbours.)

5 How to Measure Performance

- Report your hardware: CPU model, physical and logical cores, cache sizes, memory, operating system, and compiler or Java version.
- Time only the simulation loop, not setup or output (except in D5).
- Repeat each measurement at least 5 times and report the median and the spread.
- Close other heavy programs, and use the same machine for all experiments.
- Use scripts, so that all experiments can be repeated.

A Pseudocode for One Day

```
simulateDay(cur, next, day, betaEff):
    for k = 0..8: P[k] = 1 - (1 - betaEff)^k
    futures = []
    for each chunk of the grid:
        futures.add(submit(updateChunk(chunk, cur, next, day, P)))
    totals = 0
    for each f in futures:
        totals += f.get()          // waits for the task to finish
    swap(cur, next)
    return totals

updateChunk(chunk, cur, next, day, P):
    for each cell (i, j) in chunk:
        id = i*N + j
        if cur[i][j] is S:
            k = number of infected neighbours in cur
            if rand01(seed, day, id, 0) < P[k]: next[i][j] = I with c = 0
            else:                               next[i][j] = S
        else if cur[i][j] is I with counter c:
            if c + 1 == d:
                if rand01(seed, day, id, 1) < mu: next[i][j] = D
                else:                             next[i][j] = R
            else:
                next[i][j] = I with counter c + 1
```

```
        else:
            next[i][j] = cur[i][j]
    return counts of S, I, R, D in this chunk of next
```

B Formulas

With T_1 the time on one thread, T_p the time on p threads, and f the serial fraction:

$$\begin{aligned}
\text{Speedup: } S(p) &= T_1 / T_p \\
\text{Efficiency: } E(p) &= S(p)/p \\
\text{Amdahl: } S(p) &= \frac{1}{f + (1-f)/p} \\
\text{Gustafson: } S(p) &= p - f(p-1) \\
\text{Karp–Flatt: } e(p) &= \frac{1/S(p) - 1/p}{1 - 1/p}
\end{aligned}$$

If $e(p)$ stays about the same as p grows, the limit is a fixed serial part. If it grows with p, the overhead of parallelism is increasing.